

ConfigMap

Obiettivi di Apprendimento

Al termine di questa sezione il corsista sarà in grado di:

- Descrivere il ruolo dei ConfigMap nel disaccoppiare configurazione e codice applicativo
- Creare ConfigMap tramite `--from-literal`, `--from-file`, `--from-env-file` e manifest YAML dichiarativo
- Consumare ConfigMap come **variabili d'ambiente** usando `configMapKeyRef` ed `envFrom`
- Consumare ConfigMap come **volume** montando file nel filesystem del container
- Spiegare la differenza di aggiornamento tra env vars (statiche al boot) e volume mounts (aggiornamento live con delay)
- Applicare l'opzione `immutable: true` per ridurre il carico sull'API server

Teoria e Concetti Chiave

Cos'è un ConfigMap

Un **ConfigMap** è un oggetto Kubernetes (namespace-scoped) che archivia dati di configurazione non sensibili in formato chiave-valore. Separa la configurazione dall'immagine container, permettendo di riutilizzare la stessa immagine in ambienti diversi (development, staging, production) cambiando solo il ConfigMap.

```
ConfigMap → chiavi/valori → consumato da Pod come:  
├─ variabili d'ambiente (env, envFrom)  
└─ file montati (volume)
```

Limite di dimensione: un singolo ConfigMap non può superare 1 MiB. Per configurazioni più grandi usare un PVC o un Secret.

Metodi di Creazione

Metodo	Comando	Use case
<code>--from-literal</code>	<code>oc create configmap cm-name --from-literal=KEY=value</code>	Valori singoli rapidi
<code>--from-file</code>	<code>oc create configmap cm-name --from-file=app.properties</code>	File di configurazione esistenti
<code>--from-env-file</code>	<code>oc create configmap cm-name --from-env-file=.env</code>	File <code>.env</code> con <code>KEY=value</code> per riga
YAML dichiarativo	<code>oc apply -f configmap.yaml</code>	GitOps, IaC, versionamento

Consumo come Variabili d'Ambiente

Singola chiave con `configMapKeyRef`:

```
env:
- name: DATABASE_HOST
  valueFrom:
    configMapKeyRef:
      name: db-config
      key: DB_HOST
      optional: false # Pod non si avvia se ConfigMap o chiave assenti
```

Tutte le chiavi con `envFrom`:

```
envFrom:
- configMapRef:
    name: app-config
    optional: false
- prefix: APP_ # ← tutte le chiavi prefissate con APP_
  configMapRef:
    name: feature-flags
```

⚠ Attenzione: Le variabili d'ambiente vengono lette **una sola volta** all'avvio del container. Se il ConfigMap viene aggiornato, il Pod deve essere riavviato per recepire le modifiche.

Consumo come Volume

Quando un ConfigMap è montato come volume, ogni chiave diventa un **file** nella directory di mount:

```
volumes:
- name: config-vol
  configMap:
    name: app-config
    items: # ← seleziona solo alcune chiavi
    - key: application.properties
      path: application.properties # ← nome file nel volume
      mode: 0444 # ← permessi (ottale)
```

Aggiornamento automatico: Kubernetes aggiorna i file montati in ~1 minuto (basato su `kubelet syncFrequency`) quando il ConfigMap viene modificato. L'applicazione deve però **rilevare il cambiamento** (es. con inotify, polling) per ricaricare la configurazione senza riavvio.

Differenza Comportamentale: env vars vs volume

Caratteristica	env/envFrom	Volume mount
Aggiornamento dopo modifica ConfigMap	Richiede riavvio Pod	Automatico (~1 min)
Visibilità nel processo	<code>\$ENV_VAR</code> standard	File nel filesystem
Granularità	Singola chiave o tutte	Selezione con <code>items</code>
Casi d'uso	Variabili semplici, URL, flag	File di config (application.properties, nginx.conf)
Limite dimensione per chiave	~32 KiB pratico	1 MiB totale ConfigMap

Immutability

Il campo `immutable: true` blocca ogni modifica al ConfigMap (occorre cancellarlo e ricrearlo):

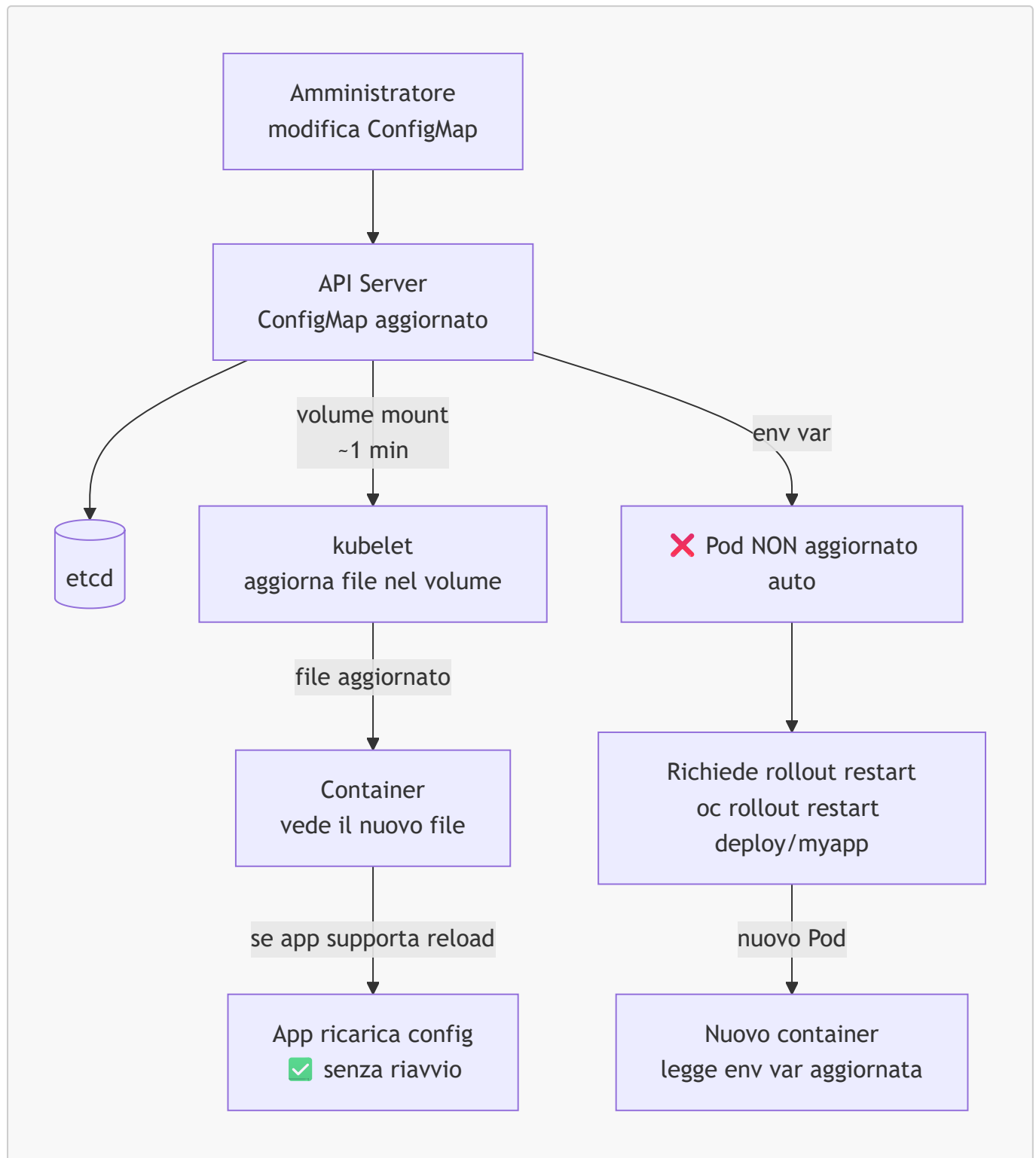
```
apiVersion: v1
kind: ConfigMap
metadata:
  name: static-config
immutable: true # ← nessuna modifica consentita dopo la creazione
data:
  ENV: production
```

Benefici su cluster grandi:

- Kubernetes smette di monitorare le modifiche a questo ConfigMap (nessun watch sull'API server)
- Riduce il carico del kubelet e dell'API server in cluster con migliaia di Pod

Diagrammi

Flusso di Aggiornamento ConfigMap: Volume vs Env Var



Comandi Pratici con Output Atteso

```
# Creare un ConfigMap con valori letterali
oc create configmap app-config \
  --from-literal=DB_HOST=postgres-svc \
  --from-literal=DB_PORT=5432 \
  --from-literal=LOG_LEVEL=info \
  -n myproject

# Output atteso:
# configmap/app-config created
```

```
# Creare un ConfigMap da file
echo "server.port=8080
spring.datasource.url=jdbc:postgresql://postgres-svc:5432/appdb" >
application.properties

oc create configmap spring-config \
  --from-file=application.properties \
  -n myproject

# Output atteso:
# configmap/spring-config created
```

```
# Visualizzare il contenuto del ConfigMap
oc get configmap app-config -n myproject -o yaml

# Output atteso (estratto):
# apiVersion: v1
# data:
#   DB_HOST: postgres-svc
#   DB_PORT: "5432"
#   LOG_LEVEL: info
# kind: ConfigMap
# metadata:
#   name: app-config
#   namespace: myproject
```

```
# Elencare tutti i ConfigMap nel namespace
oc get configmap -n myproject

# Output atteso:
# NAME          DATA  AGE
# app-config    3      2m
```

```
# spring-config      1      1m
# kube-root-ca.crt   1      10d
```

```
# Aggiornare un valore nel ConfigMap (YAML dichiarativo)
oc patch configmap app-config \
  --type merge \
  -p '{"data":{"LOG_LEVEL":"debug"}}' \
  -n myproject
```

```
# Output atteso:
# configmap/app-config patched
```

```
# Verificare che il ConfigMap sia stato aggiornato
oc get configmap app-config -o jsonpath='{.data.LOG_LEVEL}' -n myproject

# Output atteso:
# debug
```

```
# Forzare il riavvio del Deployment per recepire le nuove env vars
oc rollout restart deployment/myapp -n myproject

# Output atteso:
# deployment.apps/myapp restarted
```

```
# Descrivere i volumi montati da un Pod per verificare il ConfigMap
oc describe pod -l app=myapp -n myproject | grep -A5 "Volumes:"

# Output atteso:
# Volumes:
#   config-vol:
#     Type:      ConfigMap (a volume populated by a ConfigMap)
#     Name:      spring-config
#     Optional:  false
```

```
# Verificare il file montato all'interno del container
oc exec deployment/myapp -n myproject -- cat /config/application.properties

# Output atteso:
# server.port=8080
# spring.datasource.url=jdbc:postgresql://postgres-svc:5432/appdb
```

```
# Cancellare un ConfigMap
oc delete configmap app-config -n myproject

# Output atteso:
# configmap "app-config" deleted
```

Esempi YAML Completì

ConfigMap con Dati Multi-riga + Pod che lo Consuma

```
# ConfigMap con file di configurazione multi-riga e valori semplici
apiVersion: v1
kind: ConfigMap
metadata:
  name: myapp-config          # ← nome del ConfigMap
  namespace: myproject
  labels:
    app: myapp
    environment: production
data:
  # Valori semplici chiave-valore
  DB_HOST: "postgres-svc"    # ← hostname del servizio database
  DB_PORT: "5432"            # ← porta PostgreSQL (stringa, non intero)
  LOG_LEVEL: "info"          # ← livello di log dell'applicazione
  FEATURE_CACHE_ENABLED: "true"

  # File di configurazione multi-riga (chiave = nome file)
  application.properties: |  # ← carattere | per blocco letterale YAML
    server.port=8080
    server.servlet.context-path=/api
    spring.datasource.url=jdbc:postgresql://${DB_HOST}:${DB_PORT}/appdb
    spring.datasource.driver-class-name=org.postgresql.Driver
    logging.level.root=INFO
    management.endpoints.web.exposure.include=health,info,metrics

  nginx.conf: |              # ← configurazione nginx come file
    server {
      listen 8080;
      server_name _;
      location / {
        proxy_pass http://localhost:8090;
      }
    }
  ---
# Pod che consuma il ConfigMap sia come env vars che come volume
apiVersion: v1
kind: Pod
metadata:
  name: myapp-demo
  namespace: myproject
  labels:
    app: myapp
spec:
  containers:
    - name: app
      image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
      command: ["/bin/sh", "-c", "env | grep -E 'DB_|LOG_|FEATURE_' && cat
```



```

/config/application.properties && sleep 3600"]
resources:
  requests:
    memory: "128Mi"
    cpu: "100m"
  limits:
    memory: "256Mi"
    cpu: "500m"
env:
# Singole chiavi dal ConfigMap
- name: DB_HOST
  valueFrom:
    configMapKeyRef:
      name: myapp-config          # ← nome ConfigMap
      key: DB_HOST                # ← chiave specifica
- name: DB_PORT
  valueFrom:
    configMapKeyRef:
      name: myapp-config
      key: DB_PORT
envFrom:
# Importa tutte le chiavi semplici (non i file multi-riga)
- configMapRef:
    name: myapp-config
volumeMounts:
- name: config-files              # ← riferimento al volume
  mountPath: /config              # ← directory di mount nel container
  readOnly: true                  # ← sola lettura raccomandata
volumes:
- name: config-files
  configMap:
    name: myapp-config            # ← ConfigMap da montare
    items:                        # ← seleziona solo i file multi-riga
      - key: application.properties
        path: application.properties # ← nome file nel volume
        mode: 0444                  # ← permessi lettura per tutti
      - key: nginx.conf
        path: nginx.conf
        mode: 0444

```

ConfigMap Immutabile per Feature Flags di Produzione

```

# ConfigMap immutabile: ideale per configurazioni che non devono cambiare dopo il
deploy
apiVersion: v1
kind: ConfigMap
metadata:
  name: feature-flags-v1-2-0      # ← versione nel nome per gestione GitOps
  namespace: myproject
  labels:

```

```
  app: myapp
  version: "1.2.0"
immutable: true                # ← blocca qualsiasi modifica; richiede
delete + recreate
data:
  FEATURE_DARK_MODE: "true"    # ← feature flags dell'applicazione
  FEATURE_BETA_API: "false"
  FEATURE_ANALYTICS: "true"
  MAX_CONNECTIONS: "100"
  CACHE_TTL_SECONDS: "3600"
```

Note Specifiche per Azure ARO

Uso di ConfigMap per Configurazioni Spring Boot su ARO

Su ARO, un pattern comune è montare il ConfigMap come volume in `/config` per applicazioni Spring Boot. Spring Boot legge automaticamente `application.properties` dalla directory corrente o da `/config`:

```
# Creare ConfigMap da file application.properties esistente
oc create configmap spring-app-config \
  --from-file=application.properties=./config/application-aro.properties \
  -n myproject
```

Aggiornamento Live per Applicazioni con Reload

Su ARO, le applicazioni che supportano il reload della configurazione (es. Quarkus con `quarkus.config.source.yaml.reload`) possono sfruttare l'aggiornamento automatico dei volume mount:

 **Suggerimento:** Usare ConfigMap montati come volume (non env vars) per applicazioni che supportano hot-reload della configurazione, evitando downtime durante aggiornamenti di configurazione frequenti.

ConfigMap per Configurazione OpenShift-Specifica

Su ARO, alcune configurazioni specifiche dell'ambiente (endpoint interni Azure, nomi delle risorse) vengono spesso iniettate nei Pod tramite ConfigMap generati da Operator o da pipeline CI/CD:

```
# Creare un ConfigMap con endpoint ARO specifici dell'ambiente
oc create configmap aro-endpoints \
  --from-literal=AZURE_KEYVAULT_URI=https://mykeyvault.vault.azure.net/ \
  --from-literal=AZURE_SERVICEBUS_HOST=mybus.servicebus.windows.net \
  -n myproject
```

Dimensioni e Limiti su ARO

 **Attenzione:** Su ARO, come su qualsiasi cluster Kubernetes, un ConfigMap non può superare **1 MiB** di dati totali. Per file di configurazione di grandi dimensioni (es. certificati CA multipli, dataset statici), considerare l'uso di un PVC o di una soluzione di storage esterna.

```
# Verificare la dimensione di un ConfigMap
oc get configmap myapp-config -o json | wc -c
# Se >1048576 bytes → ridurre o suddividere il ConfigMap
```

Riepilogo dei Punti Chiave

- Un ConfigMap separa la configurazione dall'immagine container, permettendo il riutilizzo della stessa immagine in ambienti diversi
 - I dati possono essere consumati come **variabili d'ambiente** (statiche al boot del container) o come **file montati tramite volume** (aggiornamento automatico in ~1 minuto)
 - `envFrom.configMapRef` importa tutte le chiavi come env vars; `env[].valueFrom.configMapKeyRef` seleziona una chiave specifica
 - Dopo la modifica di un ConfigMap, i Pod che usano env vars richiedono `oc rollout restart`; i Pod che usano volume mount non richiedono riavvio (se l'app supporta reload)
 - `immutable: true` blocca ogni modifica e riduce il carico sull'API server nei cluster con molti Pod
 - Il limite di dimensione di un ConfigMap è 1 MiB; per file più grandi usare PVC o storage esterno
-

Domande di Verifica

1. Qual è la differenza principale tra consumare un ConfigMap come variabile d'ambiente e come volume?

- A) Le variabili d'ambiente supportano valori multi-riga, i volumi no
- B) I volumi vengono aggiornati automaticamente quando il ConfigMap cambia; le variabili d'ambiente richiedono il riavvio del Pod ☒
- C) I volumi possono contenere solo dati binari, le variabili d'ambiente solo testo
- D) Non c'è differenza: entrambi vengono aggiornati automaticamente

2. Quale campo YAML rende un ConfigMap non modificabile dopo la creazione?

- A) `readOnly: true`
- B) `locked: true`
- C) `immutable: true` ☒
- D) `frozen: true`

3. Un Deployment ha `envFrom.configMapRef` che punta a `app-config`. L'amministratore aggiorna una chiave nel ConfigMap. Come recepiscono i Pod la modifica?

- A) I Pod aggiornano automaticamente le env vars in ~1 minuto
- B) I Pod devono essere riavviati (es. `oc rollout restart`) ☒
- C) Le env vars vengono aggiornate solo al successivo scaling
- D) Il Deployment rileva automaticamente il cambiamento e fa un rolling update

4. Con `--from-file=app.properties`, qual è il nome della chiave creata nel ConfigMap?

- A) Il contenuto del file
- B) `file`
- C) `app.properties` ☒
- D) Il percorso assoluto del file

5. Quale affermazione sul campo `items` nella definizione di volume ConfigMap è corretta?

- A) `items` è obbligatorio: senza di esso il volume non viene creato
- B) `items` permette di selezionare solo alcune chiavi del ConfigMap e rinominare i file risultanti ☒
- C) `items` può contenere solo file con estensione `.properties`
- D) `items` è disponibile solo se il ConfigMap è `immutable: true`